Got it — you want to call a **native C/C++ DLL function** from **C#** using `DllImport`, where the function returns an **array of structs** and also returns the **count** via a pointer parameter.

The **safe and common pattern** for this is:

- The native function allocates or fills an array and sets the count via a pointer.
- In C#, you call it, read the count, and then marshal the array manually.

Example

Native C/C++ DLL code

```
// mylib.h
#ifdef _WIN32
#define API __declspec(dllexport)
#else
#define API
#endif

typedef struct {
    int id;
    double value;
} MyStruct;

// Example: returns pointer to static array (for demo purposes)
API const MyStruct* GetData(int* count) {
    static MyStruct data[3] = {
```

```
        {1, 10.5},
        {2, 20.75},
        {3, 30.125}
    };
    if (count) {
        *count = 3;
    }
    return data; // pointer to first element
}
```

Note:

- In real code, you might allocate dynamically and provide a separate `FreeData()` function for cleanup.
- Returning a pointer to a static array is fine for read-only demo purposes.

C# Interop Code

```
using System;
using System.Runtime.InteropServices;

class Program
{
    // Define the struct layout to match native memory layout
    [StructLayout(LayoutKind.Sequential)]
    public struct MyStruct
    {
```

```
    public int id;
    public double value;
}

// Import the native function
[DllImport("mylib.dll", CallingConvention = CallingConvention.Cdecl)]
private static extern IntPtr GetData(out int count);

static void Main()
{
    // Call native function
    int count;
    IntPtr ptr = GetData(out count);

    if (ptr == IntPtr.Zero || count <= 0)
    {
        Console.WriteLine("No data returned.");
        return;
    }

    // Marshal the array manually
    MyStruct[] managedArray = new MyStruct[count];
    int structSize = Marshal.SizeOf(typeof(MyStruct));

    for (int i = 0; i < count; i++)
    {
        IntPtr current = IntPtr.Add(ptr, i * structSize);
```

```
            managedArray[i] = Marshal.PtrToStructure<MyStruct>(current);
        }

        // Display results
        foreach (var item in managedArray)
        {
            Console.WriteLine($"ID: {item.id}, Value: {item.value}");
        }
    }
}
```

Key Points

- `[StructLayout(LayoutKind.Sequential)]` ensures the C# struct matches the native struct layout.
- `out int count` matches the `int* count` parameter in C.
- `IntPtr` is used to receive the pointer to the first struct.
- `Marshal.PtrToStructure<T>` is used to convert unmanaged memory into managed structs.
- If the native side allocates memory dynamically, you **must** provide a `FreeData()` function in the DLL and call it from C# after marshaling.

✅ This pattern is **safe** and works for both **static arrays** and **dynamically allocated arrays** (with proper cleanup).

If you want, I can also give you a **version where the C DLL allocates memory and C# frees it** — which is the correct approach for large or dynamic datasets.

Do you want me to prepare that version?